

Product Traceability for Coding Agents: A Controlled Evaluation Across Evolving Web Applications

Vlad Mysla

City College, San Francisco, California, USA

vmysla@mail.ccsf.edu · vlad@mysla.me

Final Version · 14 September 2026

Abstract

Coding agents can implement a product requirement correctly in one session yet lose the requirement, its rationale, or a rejected alternative in the next. Product Traceability is a repository-backed memory system designed to preserve that product history across fresh coding sessions. It captures requirement state, product decisions, rejected alternatives, changes, and declared implementation links; compiles them into human-readable documents; and projects a bounded product context into the next session, with detailed decision topics available on demand.

This study compared Product Traceability with standard Claude Code on three small HTML-and-JavaScript web applications: a Todo manager, a Calculator, and Tetris. Each application evolved through ten fresh sessions containing new requirements, reversals, conflicts, defect reports, and references to earlier decisions. Both configurations used Claude Code 2.1.269, Claude Opus 5, and its 1-million-token context window. Each application-configuration combination was repeated three times, yielding 18 primary trajectories and 180 sessions.

Relative to Control, Product Traceability was 38.5% cheaper and 38.3% faster on average across the three applications. It used 54.6% fewer agent round trips, 33.7% fewer all-model output tokens, and 8.5% fewer all-model new tokens. Externally graded success remained between 98.0% and 100%, with no recorded regressions. The system also produced a structured Product Record in every trajectory. Within this benchmark, lower cost and faster completion did not measurably reduce quality.

The experiment evaluates a complete configuration rather than isolating individual components. Its sample is too small for population-level inference, and all three applications began as compact single-file codebases. The results establish a controlled baseline for larger studies, not a universal effect size.

Keywords coding agents, agentic memory, requirements traceability, product history, cross-session context, developer productivity, controlled experiment

Table 1. Product Traceability: average observed change relative to Control

Configuration	Cost	Build time	Agent round trips	All-model output	All-model new tokens
Product Traceability	38.5% cheaper	38.3% faster	54.6% fewer	33.7% fewer	8.5% fewer

1 Introduction

Claude Code and similar systems divide development into sessions. Code and Git preserve implementation, but they do not reliably preserve why a feature exists, why an alternative was rejected, or which requirement superseded another. When the next session begins, the agent may reconstruct those facts from incomplete evidence and reverse a deliberate product choice.

The problem persists with large context windows. The Control in this study used Claude Opus 5 with a 1-million-token window, yet capacity does not guarantee retrieval of the governing fact [2]. Transcripts are organized by conversation order; software maintenance is organized by requirements, decisions, conflicts, and affected artifacts. Requirements traceability therefore offers a useful model: treat each requirement as an evolving object with an origin, lifecycle, rationale, and implementation links [4, 5].

1.1 From a living requirements document to Product Traceability

The project began with a living requirements document created during rapid AI-assisted prototyping. It expanded when pivots and reversals showed that current requirements alone were insufficient: later sessions also needed change history, rationale, rejected alternatives, and links to the implementation [1]. Product Traceability therefore maintains four human-readable repository artifacts:

- `requirements.md` records active, superseded, and unresolved product requirements.
- `change.log` records what changed over time.
- `decisions.md` records product choices, rationale, and rejected alternatives.
- `traceability-matrix.md` connects requirements to implementation artifacts and declared evidence.

The tested system automates their maintenance. Lifecycle hooks capture relevant events, a bounded auxiliary model extracts requirement and decision updates, and deterministic code regenerates the documents. Later sessions receive prioritized product context and can open detailed decision topics on demand. The original article supplies design provenance; this paper supplies the controlled performance evidence.

1.2 Research questions

The study asks three questions:

1. Can a generic, repository-backed Claude Code skill preserve evolving requirements, product decisions, rationale, rejected alternatives, and supersessions in a more explicit and inspectable form than relying only on source code, Git history, conversation state, and built-in memory?
2. Can the preserved knowledge serve both human developers and later AI coding sessions through human-readable records, stable identifiers, trace links, and selective cross-session context?
3. Relative to standard Claude Code, does the complete Product Traceability configuration improve software-development lifecycle efficiency as measured by recorded wall time, provider cost, interaction count, token use across the full model graph, and external product quality?

The experiment observes record production and end-to-end performance; it does not administer a blinded recall test.

1.3 Contributions and claim boundary

The paper contributes a persistent product-memory architecture and a controlled evaluation spanning speed, cost, interactions, all-model token use, conformance, regressions, and record utilization. It establishes that Product Traceability created a structured record while costing less and finishing faster than Control in these runs. The estimates describe this benchmark and are not population effects for larger repositories.

2 Product Traceability

2.1 Design hypotheses and implemented tactics

Product Traceability combines a product-memory architecture with working policy. Its tactics address known problems in traceability, provenance, code navigation, long-context retrieval, and interruption recovery, as well as failures observed during development.

Table 2. Design hypotheses and implemented tactics

Design need	Tactic and expected value	Evidence basis and study observable
Preserve requirement identity	Stable identifiers, lifecycle state, origin, acceptance intent, and supersession links let later sessions update one rule without confusing it with another	Requirements-traceability theory [4, 5]; Product Record audit
Preserve rationale	Decision objects record choices, evidence, and rejected alternatives so an intentional absence is not mistaken for unfinished work	Design-rationale research [7, 8]; record audit
Retain chronology and provenance	Append-only change and event histories connect a rule to its source and revision	Provenance model [13]; audit and recovery capability
Avoid foreground bookkeeping	Lifecycle hooks capture prompts, changed files, and completion events outside the main agent’s working loop	First-generation versus automated system anatomy; main-agent record edits
Extract selectively and safely	A small model proposes requirement state and at most two reusable decisions from bounded evidence; schema and evidence checks reject malformed or unsupported output	Selective-rationale and attribution research [7, 14, 15]; extraction audit
Connect intent to code	Deterministic compilation produces human documents and trace links to affected artifacts, reducing repeated repository archaeology	Maintenance-navigation studies [6, 11, 12]; post-hoc orientation calls
Retrieve a working set	A prioritized Cross-Session Product Context and on-demand decision topics expose current rules without injecting the complete archive; the tested implementation capped the context at 30 content lines	Long-context and tiered-memory research [2, 3, 9, 16]; footprint measurement

The operating policy also favors one source read, small checks, batched edits, one final verification, concise responses, and stopping when a defect report contradicts an explicit rule. Record-side failures do not block product work, and auxiliary calls remain in system totals. These sources motivate the design but do not supply an effect size; the controlled study evaluates the combined configuration.

2.2 Information preserved by the system

Product Traceability organizes history around semantic objects rather than transcript excerpts. A requirement carries a stable identifier, lifecycle state, source, acceptance intent, and artifact links. A decision records the choice, rationale, rejected alternatives, and supporting evidence. The change history and traceability matrix expose how these objects relate.

For example, the illustrative request “Show active tasks before completed tasks, and sort each group by priority” could be decomposed into REQ-007 – Active tasks precede completed tasks and REQ-008 – Tasks within each status group are ordered by priority. The identifiers let later prompts refine or supersede one rule without silently changing the other. These labels illustrate the mechanism; they are not identifiers quoted from the study dataset.

Machine-readable ledgers support deterministic processing; the four Markdown documents support people. The Cross-Session Product Context gives the next coding session a prioritized working selection, with detailed decision topics available on demand.

2.3 Write path, read path, and capture strategy

The coding agent always receives the complete task prompt. A prompt hook assigns a requirement identifier and retains a bounded operational copy; edit and completion hooks observe changed files and the final response. After a product-changing turn, a small auxiliary model converts bounded prompt and diff excerpts into requirement status and reusable decisions. It cannot browse the repository, edit files, use tools, or retain session history.

Schema validation and an evidence guard check the proposal before it enters the ledgers. Deterministic code then regenerates the human documents, discovers explicit requirement markers in source, and prepares cross-session context and topic references. The coding agent does not spend foreground turns maintaining the Product Record.

Figure 1 summarizes the resulting write and read paths.

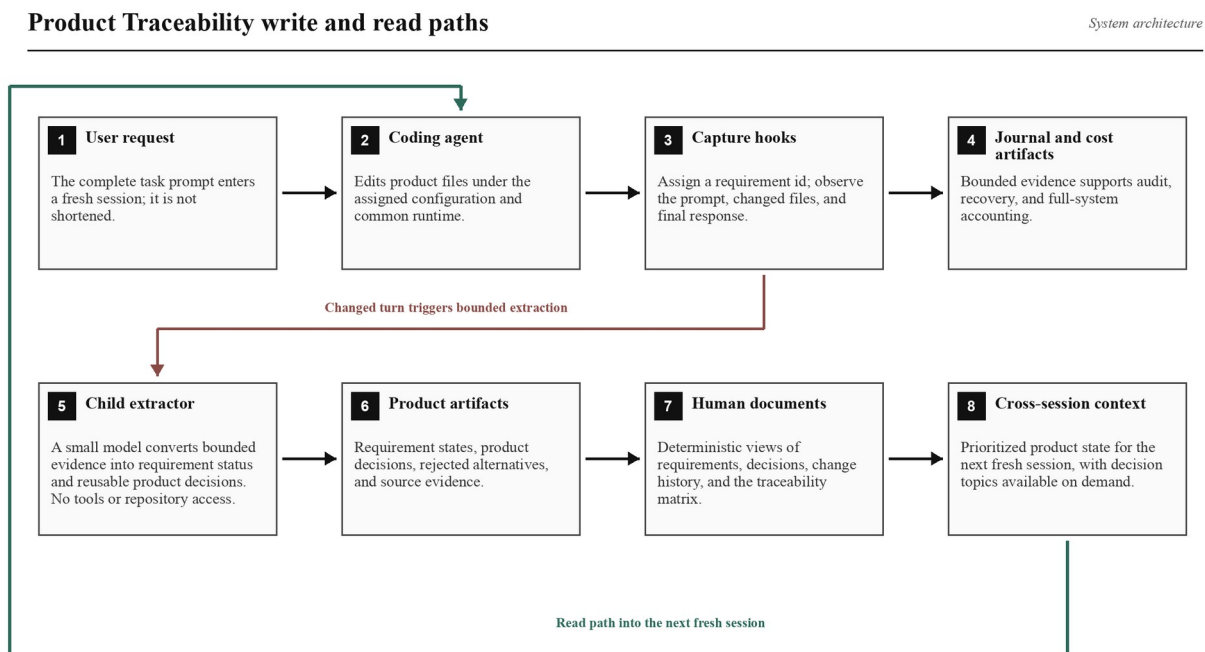


Figure 1. Product Traceability Write Path and Read Path

Hooks are lifecycle triggers; extraction is the bounded interpretation step; the Product Record is the resulting repository-backed history. Journal and cost artifacts support audit, recovery, and system accounting. They are assessment artifacts, not product requirements, and do not feed the Cross-Session Product Context.

2.4 Preservation boundaries and cap rationale

No task prompt was changed or shortened. Limits applied only to copies retained or processed by Product Traceability:

- The journal copy retained up to 4,000 prompt characters.
- The initial requirement seed retained up to 600 characters.
- The child extractor received up to 2,000 prompt characters.
- The child extractor received up to 12,000 diff characters.

These fixed values were engineering heuristics, not published best practices or optimized hyperparameters. They bound storage, extractor cost, latency, and irrelevant detail. The 600-character seed is a fail-soft placeholder normally replaced by structured extraction; the larger copies support interpretation and audit.

Across 30 unique prompts, the 600-character seed clipped 12 and retained 62.1% of source characters. The 2,000-character extractor copy and 4,000-character journal copy clipped only the first Tetris prompt, retaining 87.7% and 97.2%, respectively. The diff limit was not materially stressed. The caps were adequate for these applications but are neither optimal nor universal; larger repositories may require adaptive limits, retrieval, or chunking.

3 Methods

3.1 Experimental design and configurations

The primary experiment used three applications and three independent trajectories per application-configuration cell. Each trajectory contained ten sequential requirement batches, each in a fresh Claude Code session. The primary comparison therefore contains 18 trajectories and 180 coding sessions.

Control was standard Claude Code 2.1.269 running Claude Opus 5 with a 1-million-token context window. It had no Product Traceability plugin or added workflow policy but retained source inspection, Git history, and built-in memory. The intervention added the complete Product Traceability system.

Table 3. Primary experimental configurations

Component	Control	Product Traceability
Claude Code and model	Version 2.1.269; Claude Opus 5; 1-million-token context	Same
Added workflow guidance	None	Hook-injected Product Traceability policy
Automated capture and extraction	No	Yes
Persistent Product Record	No	Yes
Cross-session product context	Ordinary Claude Code behavior	Product Traceability context plus ordinary behavior

Both configurations received identical task batches, starting source, effort setting, permission mode, grader, and repository shape. Sources were frozen, and trajectories ran one at a time in a seeded randomized schedule.

Todo had been used during system tuning, so its reported values come only from a fresh rerun under the final protocol. Earlier exploratory Todo data were excluded. Calculator and Tetris provide frozen-configuration replication on applications not used for the main optimization loop.

3.2 Tasks and real-life simulation of evolving software work

The tasks were small HTML-and-JavaScript web applications, simple enough for deterministic grading but developed as evolving products rather than one-shot specifications. A hidden PRD for each application was divided into ten batches; the agent saw only the current batch.

The sequences approximated real product work: requirements accumulated or conflicted, features were partially reversed, defects appeared later, and some requests depended on earlier decisions without restating their rationale. These planted continuity demands tested evolving product knowledge rather than greenfield generation alone.

Table 4. Applications and simulated development pressures

Application	Checks per complete run	Product evolution simulated across ten batches
Todo manager	24	Task creation and state, date behavior, filtering, persistence, a partial reversal, defect handling, and references to earlier agreements
Calculator	33	Arithmetic rules, evaluation order, parentheses, key removal while retaining another key, history-tape behavior, formatting defects, and scientific operations
Tetris	29	Board and controls, scoring and level rules, previews, a partial preview reversal, pause and lock behavior, a defect report, and keyboard constraints

The benchmark does not reproduce every feature of professional development, but it is closer to an evolving workflow than a static prompt.

3.3 Session, clarification, and grading protocol

Every batch launched a new session. Conversation history did not carry forward, while code, Git history, repository state, and ordinary Claude Code memory remained. Control was therefore a realistic no-plugin baseline, not a memory-free condition.

The main agent always received the complete batch prompt. If it invoked `AskUserQuestion` or `SendUserMessage`, the runner logged the question and returned the same fixed response in every configuration:

Proceed according to the information already provided. Do not ask further questions.

The response supplied no hidden-PRD information or implementation direction. The same sentence served as a neutral nudge after 180 seconds of silence; a 900-second deadline protected each run. After each batch, a deterministic browser grader evaluated all live acceptance checks, the runner committed state, and the next session began. Product Traceability extraction completed inside the measured workflow, including its cost and model usage.

3.4 Outcomes and analysis

The operational outcomes were vendor-reported cost, recorded build time, agent round trips, all-model output tokens, and all-model new tokens. A round trip is a distinct assistant message after identifier deduplication. All-model output includes the primary coding agent, any in-session auxiliary model calls, and Product Traceability’s external extractor. All-model new tokens are uncached input, cache-creation input, and output across the same model graph. Cache-read tokens were retained in the source analysis but are not counted as new compute.

Recorded build time is the sequential end-to-end wall time captured by the harness for a complete ten-batch trajectory. A positive speed improvement means the configuration used less mean wall time than Control:

$$\text{speed improvement} = 100 \times (1 - \text{configuration mean} \div \text{Control mean})$$

Final conformance is the percentage of externally graded checks passing after batch ten. A regression is a previously passing, still-live criterion that later failed. Questions and timeouts were counted directly from runner events.

Product Memory Utilization describes byte footprint. Its denominator includes machine-readable requirement and decision ledgers plus the four human documents, but excludes journals, cost telemetry, topic files, projection metadata, and transient state. Context and human-document percentages do not measure semantic completeness or causal token savings. Automated test generation, code coverage, and requirement-to-test linkage were outside scope; an external grader assessed behavior.

Each application-configuration result is the mean of three trajectories. Percentages compare Product Traceability with the corresponding Control mean. Cross-application values are equal-weighted means of the three application-level reductions. With only three trajectories per cell, bootstrap intervals are descriptive and exact permutation tests lack useful resolution. The paper therefore reports observed means and consistency without claims of statistical significance, equivalence, or noninferiority [19, 20].

4 Product Traceability Light as a Derived Configuration

4.1 Derivation and scope

Optimization evidence suggested that several behavioral tactics could be expressed as static instructions: reading source once, keeping temporary checks small, batching edits, verifying once, handling explicit product rules consistently, and ending concisely. Those lessons were distilled into **Product Traceability Light**, and the resulting prompt was frozen for evaluation.

Light was a later derivative, not the plugin with features disabled. It has no hooks, child extraction, Product Record, traceability matrix, or projected product context. It was evaluated under the same runtime, tasks, grader, and three-run protocol, adding nine trajectories and 90 sessions. The full dataset therefore contains 27 trajectories and 270 sessions.

4.2 Product Traceability Light prompt

The frozen prompt is reproduced verbatim below.

Working rules

1. When you need the source, read it once with the Read tool. Do not page through it with sed, and do not list the directory.
2. Checks are optional and small: a few assertions on pure functions in `check/`, run with node. Never build test infrastructure inside a task: no stubs, no frameworks, no simulated environments. If a behaviour cannot be asserted in a few lines, do not write a check for it.
3. Send all the edits a task needs in ONE message. Verify once, at the end, not after every edit. If asked to verify earlier work, run the checks once and fix only what is red.
4. If a report contradicts a rule the project has stated and the code follows the rule, say the report cannot be reproduced under that rule and stop; do not investigate around it.
5. End with a final message of at most three short lines: what you chose, why, and what you rejected. Only rules and rejected alternatives a later session would need; no narration of what you did.
6. Rules the project has stated persist until a prompt explicitly cancels one. If a new request seems to conflict with a rule, satisfy the request within the rule; never drop a rule silently. The reverse also holds: never remove existing behaviour to satisfy a rule unless the current prompt asks for it. If the code and a rule disagree, say so in your final message and leave the code.

4.3 Observed outcomes and recommendation

Relative to Control, Light recorded favorable movement in every operational measure in every application. It averaged 40.7% lower cost, 49.6% less build time, 50.5% fewer round trips, 44.6% fewer output tokens, and 28.5% fewer new tokens. It passed 257 of 258 external checks and recorded no regressions. Because it created no persistent Product Record, a record layer was not necessary to observe these gains in this benchmark; the design does not isolate the contribution of any one rule.

Table 5. Product Traceability Light relative to Control

Metric	Todo	Calculator	Tetris	Average
Cost	35.2% cheaper	40.8% cheaper	46.0% cheaper	40.7% cheaper
Build time	51.7% faster	42.8% faster	54.3% faster	49.6% faster
Agent round trips	39.6% fewer	54.3% fewer	57.5% fewer	50.5% fewer
All-model output tokens	44.7% fewer	38.0% fewer	51.3% fewer	44.6% fewer
All-model new tokens	25.2% fewer	27.8% fewer	32.6% fewer	28.5% fewer

For small, short-lived applications similar to those tested—such as prototypes, utilities, and experiments—Light is the directly supported recommendation. It offers the six observed workflow constraints with almost no runtime machinery. Products that need durable requirements, rationale, handoffs, or auditability are the intended use case for the complete Product Traceability system.

Figure 2 renders the final run-01 artifact for every application and configuration under the same representative UI state. These post-study captures document the implemented interfaces; they were not used as quality ratings.

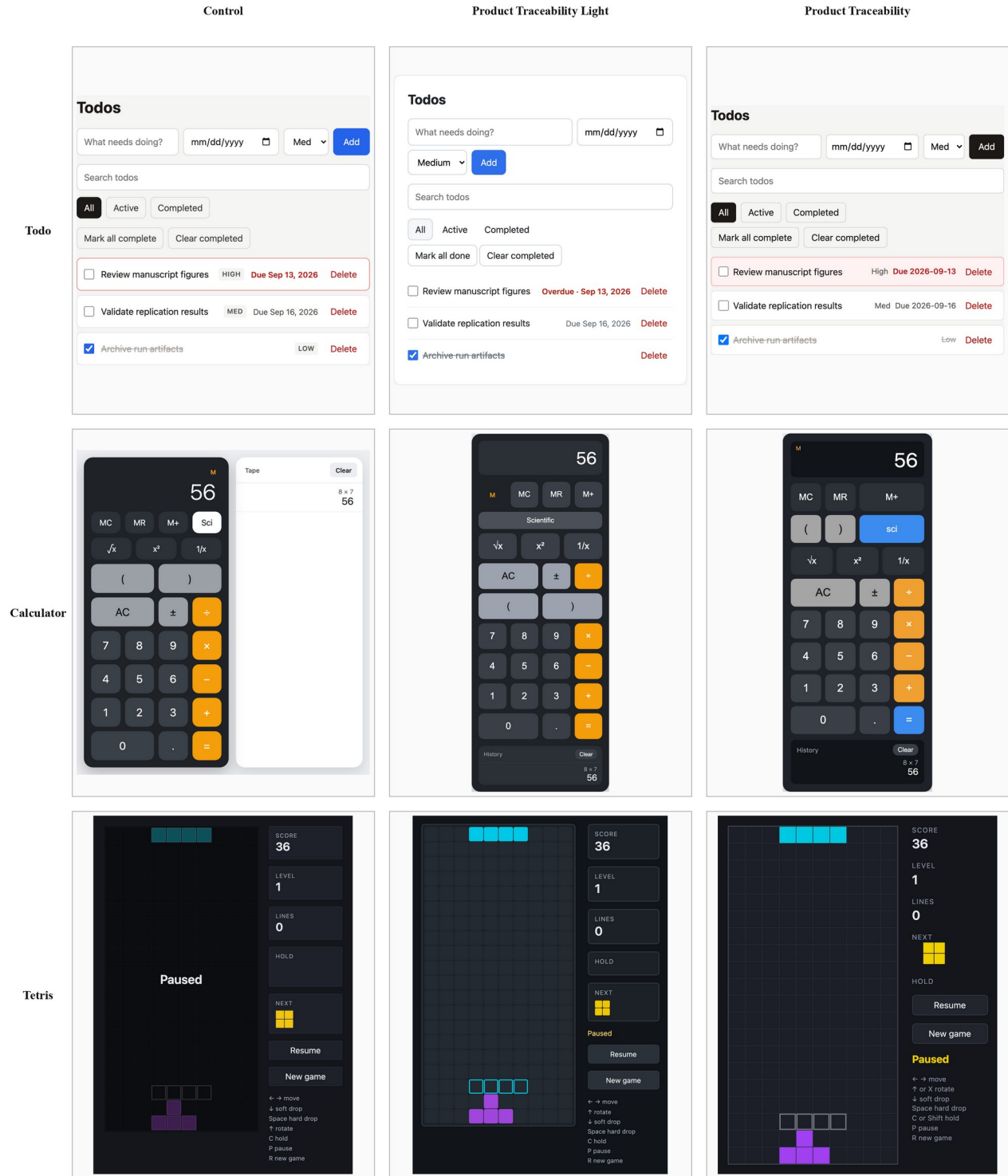


Figure 2. Final Applications from the First Replication Run

5 Product Traceability Results

5.1 Operational outcomes

Product Traceability recorded favorable movement in every measured operational outcome. Across applications it averaged 38.5% lower cost, 38.3% less build time, 54.6% fewer agent round trips, 33.7% fewer all-model output tokens, and 8.5%

fewer all-model new tokens. Cost, wall time, and all-model token totals include external extraction; agent round trips count main-session assistant messages.

Figure 3 reports the application-level reduction in recorded build time.

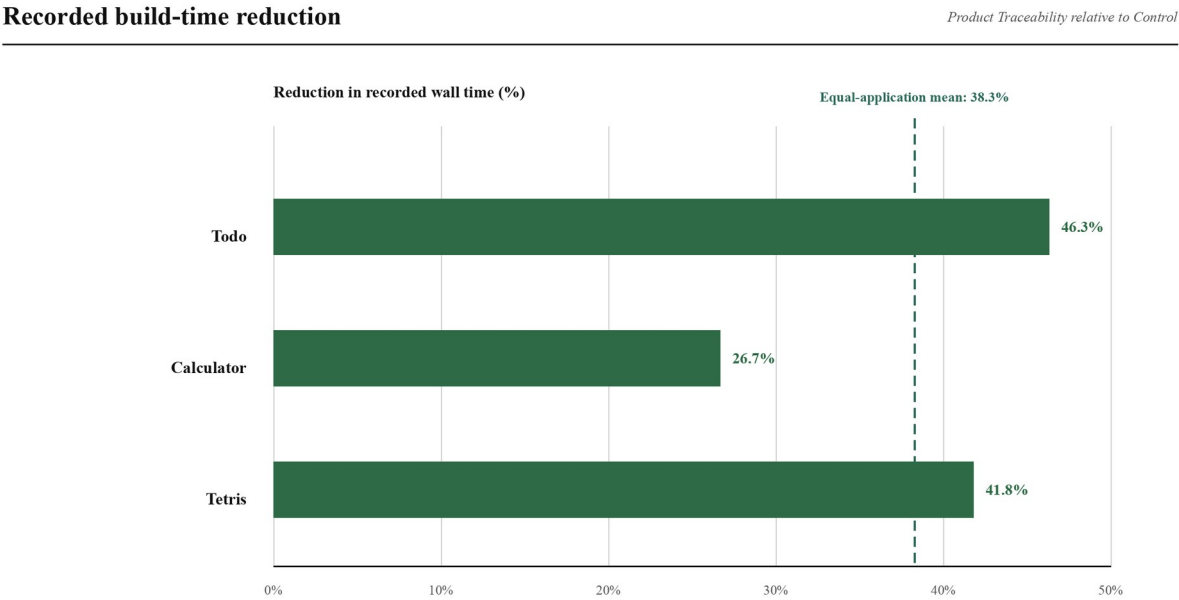


Figure 3. Product Traceability Development Speed

Table 6. Product Traceability relative to Control

Metric	Todo	Calculator	Tetris	Average
Cost	39.4% cheaper	33.8% cheaper	42.2% cheaper	38.5% cheaper
Build time	46.3% faster	26.7% faster	41.8% faster	38.3% faster
Agent round trips	51.7% fewer	52.0% fewer	60.2% fewer	54.6% fewer
All-model output tokens	39.9% fewer	24.2% fewer	37.0% fewer	33.7% fewer
All-model new tokens	8.5% fewer	4.6% fewer	12.4% fewer	8.5% fewer

The differences apply to the complete configuration. They do not allocate improvement among policy, hooks, extraction, context delivery, or source mapping.

Figure 4 summarizes the equal-application reductions in operational resource use.

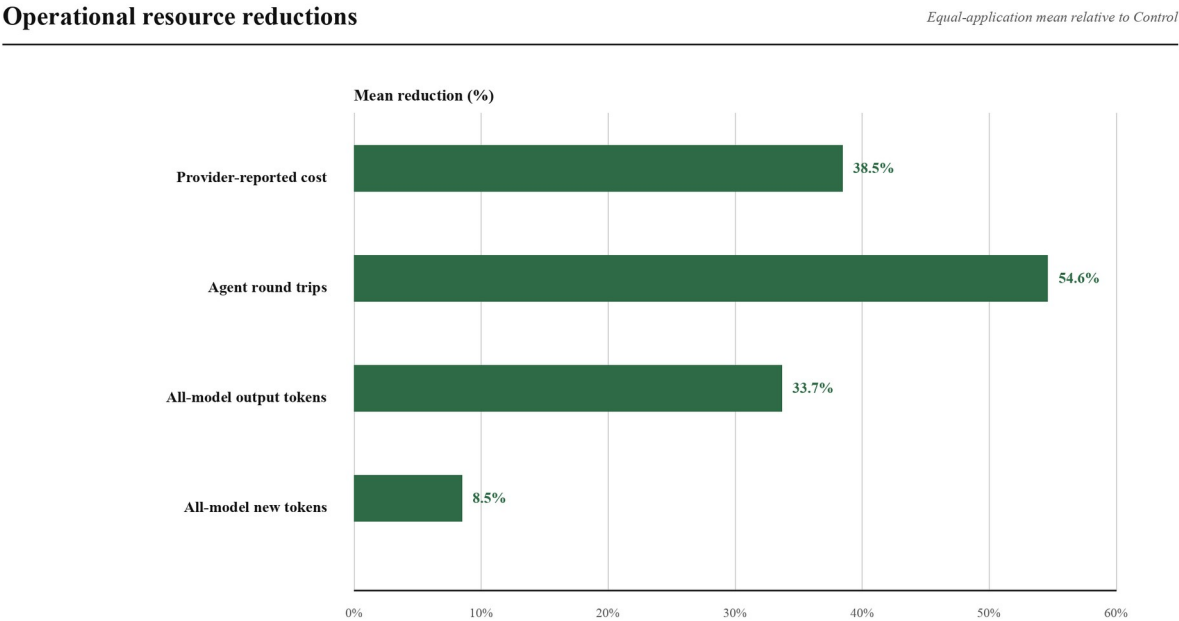


Figure 4. Product Traceability Operational Resource Use

5.2 External product quality

The applications had different acceptance suites, so Table 7 reports their denominators and success rates.

Table 7. Final external acceptance results

Application	Checks per complete run	Runs per configuration	Control success	Product Traceability Light success	Product Traceability success
Todo	24	3	100%	100%	100%
Calculator	33	3	100%	100%	98.0%
Tetris	29	3	100%	98.9%	100%

Success remained between 98.0% and 100%, and no regressions were recorded. Within this near-ceiling grader and three-run cells, the study found no evidence of a meaningful quality reduction. This is not a formal equivalence or noninferiority result; isolated misses may be noise or real edge cases that the sample cannot resolve.

5.3 Product Memory Utilization

Every Product Traceability trajectory created machine-readable ledgers, human-readable product documents, and a Cross-Session Product Context. The complete selected Product Record is defined as 100%.

Table 8. Product Memory Utilization by application

Application	Complete Product Record	Human-readable product documents	Cross-Session Product Context	Completed requirements with declared implementation link
Todo	100%	43.2%	10.6%	100%
Calculator	100%	42.0%	9.4%	100%
Tetris	100%	41.8%	9.4%	100%
All combined	100%	42.3%	9.8%	100%

Figure 5 compares the human-readable document and cross-session context footprints across applications.

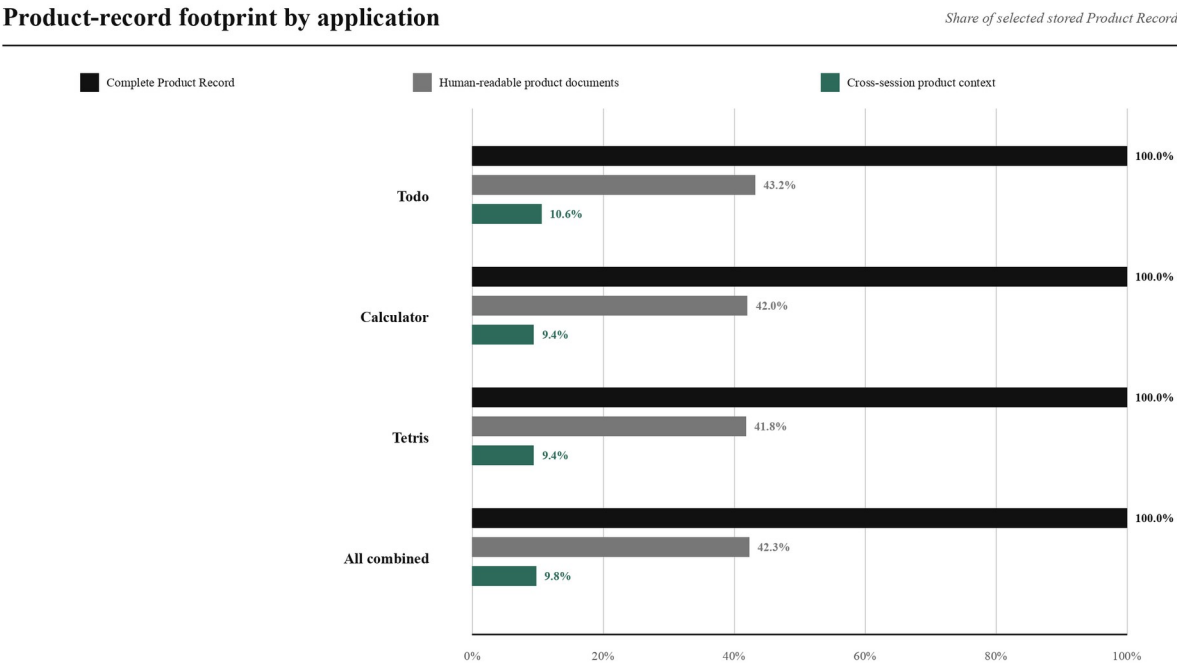


Figure 5. Product Memory Utilization

The percentages describe storage shape, not semantic retention or causal token savings. The Product Record reached the next session through a smaller prioritized index, with detailed rationale available on demand. Implementation links are declared traceability rather than proof of correctness.

Control retained source, Git, and ordinary Claude Code memory; native project-memory files appeared in six of nine Control trajectories. It had no equivalent ledger, compiler, traceability matrix, or projected context. Product Memory Utilization is therefore undefined for Control rather than 0% because the denominator exists only for Product Traceability.

5.4 Evidence about mechanisms

The controlled experiment compared complete configurations, not individual tactics. Component evidence comes from the sequential Todo optimization loop and post-hoc transcript decomposition, so Table 9 explains plausible mechanisms without assigning causal shares. In the table, “turns” reproduce manifest-reported assistant turns from optimization runs; “calls” and category counts are deduplicated agent/tool interactions from the post-hoc analysis. These diagnostics are not the final agent-round-trip KPI.

Table 9. Mechanism evidence and interpretation boundary

Tactic	Observed evidence	Interpretation
Move record maintenance off the main loop	The manual first generation added 163 record-file calls; Todo cost rose from \$5.59 to \$16.06 and turns from 148 to 361. Final Product Traceability made no main-agent record edits	Foreground bookkeeping was expensive; off-loop maintenance is compatible with lower total cost, though the systems differed in other ways
Read source once and map requirements to artifacts	A paged-reading run used 34 product reads, \$8.04, and 127 turns; the next bundled configuration used \$5.75 and 75 turns. Final Todo orientation calls fell from 10.7 to 0.2	Repeated paging and directory exploration were removed; source mapping was not isolated
Keep checks small	One costly run created a DOM-stub harness with 15 writes; post-hoc analysis found 6.7 fewer harness calls	Disposable test infrastructure materially increased interaction on these small tasks
Capture rules with evidence guards	Without the date rule, a defect batch used 26 calls and \$1.51. User-source capture reduced it to 7 turns and \$0.43, but faulty capture later reduced quality to 21/24; the repaired configuration passed 24/24	Available rules reduced re-derivation, while unsupported durable memory created a real failure
Limit decision volume	One run extracted 47 decisions and displaced an older rule. A bounded design reduced cost from \$8.04 to \$5.75 and turns from 127 to 75	Over-capture was observed; the decision cap's standalone effect is unknown
Verify once and handle explicit conflicts	A bundled change reduced a run from \$3.47 and 50 turns to \$3.12 and 43 turns at 24/24; verification calls fell from 25.3 to 2.8	Repeated verification was the largest removed-call category, but two tactics changed together
Batch edits and preserve cross-session context	Fewer edit operations accounted for 22% of removed calls. A defect batch without the relevant decision used \$1.00 and 16 turns; an earlier Control run with a native note used \$0.33 and 8 turns	Both observations support less foreground rework, but neither isolates a causal effect

The evidence is consistent with less foreground bookkeeping, orientation, verification, and re-derivation.

6 Discussion

6.1 Interpretation of the controlled result

Product Traceability completed all three application sequences with lower cost, less recorded time, fewer agent messages, and fewer generated tokens than standard Claude Code, while creating a structured Product Record in every trajectory. Success remained at 98.0–100% and no regressions were recorded; within the study's near-ceiling grader, there was no evidence of a meaningful quality reduction. Because the configuration combined policy, capture, extraction, compilation, mapping, and context delivery, the study does not show that archival memory alone caused the gains.

6.2 Scaling and practical use

Product records will grow with their products. The important questions are whether capture remains accurate, compilation remains tractable, retrieval selects the governing material, and the system saves more work than it adds. Larger repositories may make preserved rationale more valuable, but they also increase extraction, indexing, retrieval, and stale-record costs. Three compact web applications cannot establish the net effect.

Product Traceability is designed for software that evolves across sessions or contributors and for cases where rationale, reversals, handoffs, or auditability matter. Light is better supported for small, short-lived work similar to the applications tested here. The next study will test multi-file repositories, additional languages and architectures, longer histories, and several codebase scales. It will measure retrieval precision, time to find the governing requirement, conflict handling, correction of stale decisions, end-to-end cost, and product quality.

6.3 Implications for builders of agentic coding systems and future research

Preserve intent and make it correctable

Code and Git do not reliably record why a behavior exists, why an alternative was rejected, or which requirement superseded another. Later agents can therefore reconstruct intent incorrectly. Requirements and decisions should be inspectable repository objects with stable identity, lifecycle, chronology, provenance, evidence, artifact links, conflict state, and a visible correction path [8, 9, 13–15].

Account for the complete agent system

Modern workflows distribute work across planners, subagents, extractors, retries, and post-turn jobs. Measuring only the primary agent can understate cost and latency. Every model call, token and cache class, retry, and background step should be attributed to the user-visible trajectory and included in published totals.

Retrieve context selectively and transparently

More history does not guarantee better decisions: injecting everything wastes compute, while excessive compression can hide the governing constraint. Task-aware retrieval, prioritized indexes, on-demand topics, and disclosure of what was selected can balance these risks [2, 3, 9, 16]. Evaluation should connect retrieval quality to correctness and task performance rather than optimize context size alone.

Strengthen quality and human evaluation

Near-ceiling acceptance tests may conceal rare regressions or structural defects. Operational results should be paired with independent task authorship, mutation-resistant tests, blinded review, adversarial changes, and requirement-level coverage before supporting claims of equivalence or safety. Studies should also measure interruption recovery, time to a confident first edit, cognitive load, trust in stored decisions, and the effort required to correct stale memory [10, 17, 18]. A record that performs well in a benchmark may still fail the people expected to use it.

7 Limitations

This is the first controlled A/B evaluation of Product Traceability; earlier evidence was observational, pre/post, or part of optimization. Holding runtime, model, tasks, grader, and starting source constant strengthens interpretation, but three trajectories per cell cannot establish stable population effects. The author also designed the system, tasks, and grader, so frozen sources and deterministic analysis reduce but do not remove bias.

All applications began as small, single-file HTML-and-JavaScript products; two Product Traceability Light Todo outputs later split logic into a second script. They do not represent dependency-heavy services, concurrent teams, regulated software, migrations, other languages, or book-length repositories. Todo informed system tuning before its protocol-matched rerun; Calculator and Tetris are stronger frozen-configuration checks, but three applications remain a narrow sample.

Control was no-plugin, not no-memory: source, Git, and ordinary Claude Code memory remained. Inconsistent native-memory use and the absence of a blinded recall outcome prevent a clean comparison of memory systems. The near-ceiling grader also cannot establish quality equivalence or cover every structural requirement; automated test coverage and requirement-to-test linkage were outside scope.

Finally, the mechanism analysis is not factorial. Tactics changed in bundles, and post-hoc decomposition assumes comparable call cost. Table 9 explains plausible mechanisms rather than allocating causal shares.

8 Reproducibility and Data Availability

The source and replication package have been prepared for publication in the Product Traceability repository. At finalization, the release tree existed locally but had not yet been committed to the remote repository; consequently, this paper does not claim an immutable commit, tag, or archival identifier. The public release should be cited by the tag that accompanies this final paper.

The prepared repository has the following organization:

```
agent-skill-product-traceability/
|-- SKILL.md, README.md, CITATION.cff
|-- bin/                      record, extraction, compilation, and memory tools
|-- hooks/                   Claude Code lifecycle hooks
|-- light/CLAUDE.md          Product Traceability Light prompt
|-- tests/check.sh           plugin checks
|-- ab-study/
|   |-- specs/{todo,calculator,tetris}/
|   |   |-- full-prd.md
|   |   |-- batches/01.md ... 10.md
|   |-- reference/{calculator,tetris}/index.html
|   |-- builds/
|   |   |-- product-traceability-v2.6/
|   |   |-- product-traceability-light/
|   |-- runner/
|   |   |-- replicate.py, run_arm.py
|   |   |-- grade.mjs, acceptance.{todo,calculator,tetris}.mjs
|   |   |-- measure.py, collect.py
|   |-- results/2026-09-replication/
|   |   |-- runs/<run>/<arm>/
|   |   |-- arms/<run>/<arm>/
|   |   |-- runs.csv
|   |-- summary.md
```

The package contains 27 complete manifests, 270 raw session transcripts, 270 grade files, 27 final applications, and nine Product Records. The tested full build is preserved under `ab-study/builds/product-traceability-v2.6/`; it retains the historical product-record naming used during the experiment. The tested Light source is preserved under `ab-study/builds/product-traceability-light/`. Derived tables can be recomputed from the repository root without third-party Python packages:

```
python3 ab-study/runner/collect.py --data ab-study/results/2026-09-replication
```

A complete rerun requires Git, Python 3.9 or later, Node.js 18 or later, Playwright Chromium, Claude Code 2.1.269, and access to Claude Opus 5. From `ab-study/`, the protocol is:

```
npm install
npx playwright install chromium
python3 runner/replicate.py --n 3 --seed 20260912
python3 runner/collect.py
```

The runner uses `--permission-mode bypassPermissions`; replication should therefore occur in a disposable virtual machine, container, or separate user account. Raw transcripts contain local paths. Finished arm directories omit duplicated plugin installations and their temporary Git histories, while the frozen builds and all measured session and grading evidence remain in the package.

9 Conclusion

Product Traceability converts bounded lifecycle evidence into an inspectable Product Record, human documents, and cross-session context. Across Todo, Calculator, and Tetris, it was 38.5% cheaper and 38.3% faster than standard Claude Code, with 54.6% fewer agent round trips. Success remained at 98.0–100% with no recorded regressions, providing no evidence of a meaningful quality reduction within this near-ceiling grader.

The derived Light prompt was 40.7% cheaper and 49.6% faster than Control, supporting its use on small, short-lived work similar to the applications tested here. Full Product Traceability addresses a different need: preserving rationale and changing requirements as durable product assets. The study evaluates complete configurations, not memory or individual tactics in isolation. Larger and more diverse controlled tests are required before extending either effect size beyond these applications.

References

- [1] V. Mysla. “AI Coding Skill That May Make Your Coding Sessions +44% More Productive.” *Substack*, 26 May 2026.
- [2] N. F. Liu et al. “Lost in the Middle: How Language Models Use Long Contexts.” *Transactions of the Association for Computational Linguistics*, 12, 157–173, 2024.
- [3] C. Packer et al. “MemGPT: Towards LLMs as Operating Systems.” arXiv:2310.08560, 2023.

- [4] O. C. Z. Gotel and A. C. W. Finkelstein. “An Analysis of the Requirements Traceability Problem.” *First International Conference on Requirements Engineering*, 1994.
- [5] B. Ramesh and M. Jarke. “Toward Reference Models for Requirements Traceability.” *IEEE Transactions on Software Engineering*, 27(1), 58–93, 2001.
- [6] P. Mäder and A. Egyed. “Do Developers Benefit from Requirements Traceability When Evolving and Maintaining a Software System?” *Empirical Software Engineering*, 20, 413–441, 2015.
- [7] D. Falessi et al. “The Value of Design Rationale Information.” *ACM Transactions on Software Engineering and Methodology*, 22(3), Article 21, 2013.
- [8] U. van Heesch, P. Avgeriou, and R. Hilliard. “A Documentation Framework for Architecture Decisions.” *Journal of Systems and Software*, 85(4), 795–820, 2012.
- [9] D. Wu et al. “LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.” *International Conference on Learning Representations*, 2025.
- [10] S. J. Gilbert. “Strategic Offloading of Delayed Intentions into the External Environment.” *Quarterly Journal of Experimental Psychology*, 68(5), 971–992, 2015.
- [11] A. J. Ko, H. H. Aung, and B. A. Myers. “How Developers Seek, Relate, and Collect Information during Software Maintenance.” *IEEE Transactions on Software Engineering*, 32(12), 971–987, 2006.
- [12] J. Yang et al. “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.” *Advances in Neural Information Processing Systems*, 2024.
- [13] W3C. “PROV-DM: The PROV Data Model.” W3C Recommendation, 30 April 2013.
- [14] L. Gao et al. “RARR: Researching and Revising What Language Models Say, Using Language Models.” *Proceedings of ACL*, 2023.
- [15] T. Gao, H. Yen, J. Yu, and D. Chen. “Enabling Large Language Models to Generate Text with Citations.” *Proceedings of EMNLP*, 2023.
- [16] D. Wu et al. “Repoformer: Selective Retrieval for Repository-Level Code Completion.” *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [17] N. Forsgren et al. “The SPACE of Developer Productivity: There’s More to It Than You Think.” *ACM Queue*, 19(1), 2021.
- [18] A. Noda, M.-A. Storey, N. Forsgren, and M. Greiler. “DevEx: What Actually Drives Productivity.” *ACM Queue*, 21(2), 2023.
- [19] B. A. Kitchenham et al. “Preliminary Guidelines for Empirical Research in Software Engineering.” *IEEE Transactions on Software Engineering*, 28(8), 721–734, 2002.
- [20] D. Lakens. “Sample Size Justification.” *Collabra: Psychology*, 8(1), 33267, 2022.